

# **NDNSF: A Data-Centric Service Framework for Secure, Collaborative, and Dynamic Networked Applications**

Tianxing Ma

Department of Computer Science  
University of Memphis  
Advisor: Dr. Lan Wang

PhD Proposal Defense  
June 2026

# What is a service framework?

**A service framework is a set of application-layer protocols and libraries that turns distributed functions into reusable services.**

- ▶ It defines how applications discover, authorize, invoke, and compose services.
- ▶ It hides provider location and implementation details behind service interfaces and policies.
- ▶ It manages application concerns such as provider readiness, selection, retries, security, and data transfer.
- ▶ It is useful in UAV control, edge AI inference, storage, sensing, robotics, and many other applications.

# Issues with IP-Based Service Frameworks

- ▶ **Endpoint-oriented invocation:** service calls commonly resolve to reachable hosts, processes, brokers, or endpoint abstractions.
- ▶ **Channel/session-centered security:** TLS protects communication channels; stored or cached objects need separate object-level protection.
- ▶ **Service semantics outside invocation:** identity, trust, network reachability, and service discovery are configured outside the service invocation.
- ▶ **Fundamental issue:** mismatch between IP's host-centric model and service-oriented applications' data-centric needs.

Existing tools solve pieces of the problem; applications still bridge hosts, services, trust, and data by hand.

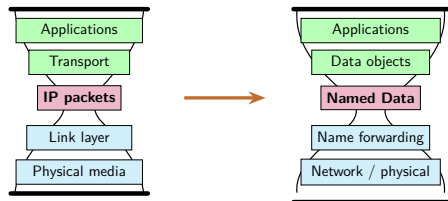
# Proposal Defense Roadmap

1. **Named Data Networking (NDN)** [1]: background and motivation.
2. **Research gap**: what service-layer semantics are missing.
3. **Framework**: NDN Service Framework (NDNSF) core design and completed foundation.
4. **Workloads**: UAV network application and distributed inference as validation cases.
5. **Plan**: evaluation matrix, remaining work, risks, and timeline.

The central claim is that NDNSF is the dissertation contribution; the applications validate and refine the framework.

# Why NDN Helps

NDN [1] changes the narrow waist from host-addressed packets to named, secured data chunks.



| IP-centered issue                                | NDN mechanism                                                   |
|--------------------------------------------------|-----------------------------------------------------------------|
| Calls resolve to hosts or endpoints              | Interests retrieve named Data from any valid producer or cache. |
| Security protects a channel                      | Every Data packet carries a verifiable signature.               |
| Failover and mobility need endpoint repair       | Name-based forwarding can use another path or producer.         |
| Objects need separate storage/distribution logic | Named Data can be cached, stored, and fetched unchanged.        |

NDN supplies named retrieval and data-centric security; NDNSF adds the missing service semantics: authorization, readiness, invocation, and provider coordination.

# NDN Packet Model

## Interest Packet

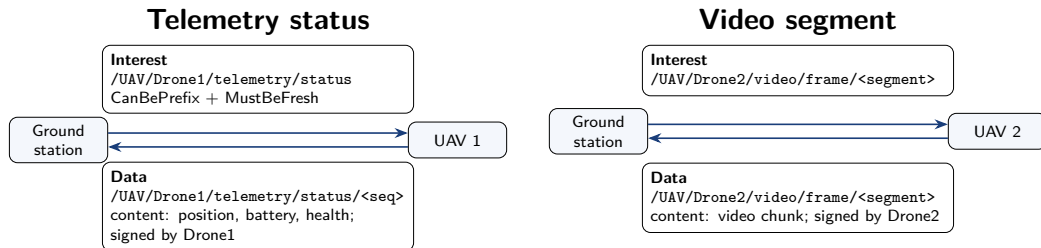
| Name                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------|
| Optional Fields<br>CanBePrefix, MustBeFresh,<br>ForwardingHint, Nonce,<br>InterestLifetime, HopLimit,<br>Parameters, ... |

## Data Packet

| Name                                                             |
|------------------------------------------------------------------|
| MetaInfo<br>ContentType, FreshnessPeriod, ...                    |
| Content                                                          |
| Signature<br>signature type, key locator,<br>signature bits, ... |

In NDN [1], Interests name desired Data; Data packets carry names, content, metadata, and signatures.

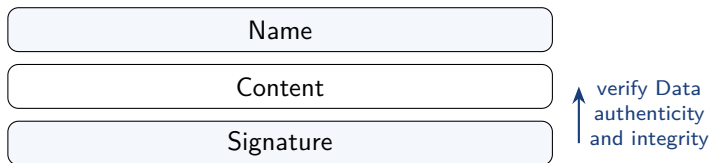
# NDN Packet Example: UAV Telemetry and Video



**Takeaway:** applications name desired data; any valid producer, cache, or repository can return signed Data with a matching name.

# NDN Data-Centric Security

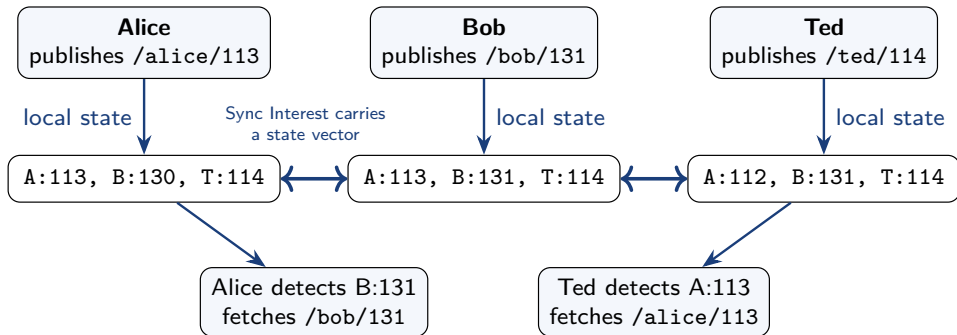
In NDN [1], security is attached to Data, not to the channel that carried it.



- ▶ A consumer can verify Data after retrieval.
- ▶ Verification still works through caches, repositories, and alternate paths.
- ▶ Access-control policy is a higher-layer question, to be explained later with NDNSF.

# NDN Sync and State Vector Sync

NDN Sync [2] communicates dataset changes; State Vector Sync (SVS) [3] synchronizes Alice, Bob, and Ted's knowledge of that dataset.

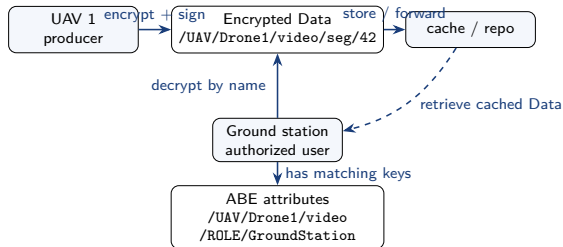


Each participant compares received and local sequence numbers, identifies missing names, and fetches the corresponding Data. All three converge on the same dataset without a central broker.

# Name-Based Access Control: NAC

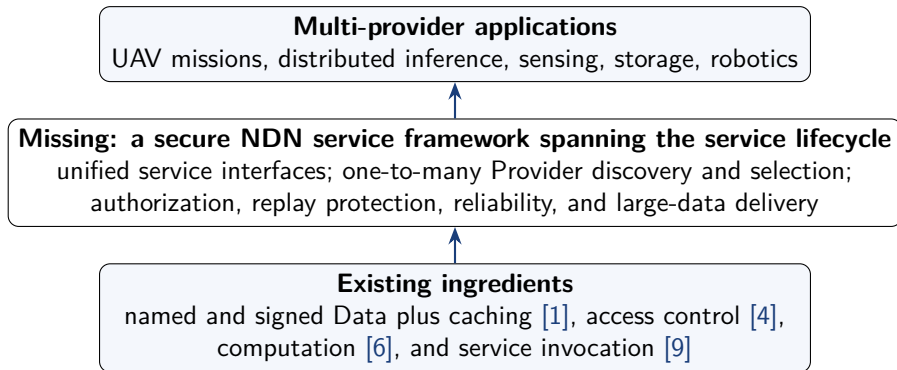
**NAC [4]** protects named NDN Data; **NAC-ABE [5,19]** applies attribute-based decryption policy.

- ▶ Content is encrypted at production, so Data stays protected in caches and repositories.
- ▶ NDN names express policy granularity and key names for automatic key retrieval.
- ▶ NAC-ABE uses attributes to decide which identities can decrypt named Data.



Example: UAV telemetry or video Data can be cached anywhere, but only ground stations with matching NAC-ABE attributes can decrypt it.

# Research Gap



NDNSF v0.1 supplies the core framework functions; the proposed dissertation work extends them to multi-role collaboration and verifies Data flow between assigned roles.

# Related Work: NDN Service Framework Directions

**Computation / composition:** NFN [6], NFaaS [7], CFEC [8], CFN [14], CaSCON [15], and data-centric composition [21]

Named functions, placement, computation graphs, and dependency-aware composition.

**Content authorization:** NAC [4], NAC-ABE [5,19], Trust Schema [16], and access-controlled derived Data [20]

Protect content and constrain valid signers; policy is not derived from a runtime collaboration plan.

**Invocation:** RICE [9], NSC [10]

Named remote calls and execution; do not by themselves enforce a late-bound multi-role Data-admission policy.

**Application-specific:** MIA-NDN [11], DNMP [12], SECaaS [13]

Targets aggregation, measurement, or edge security.

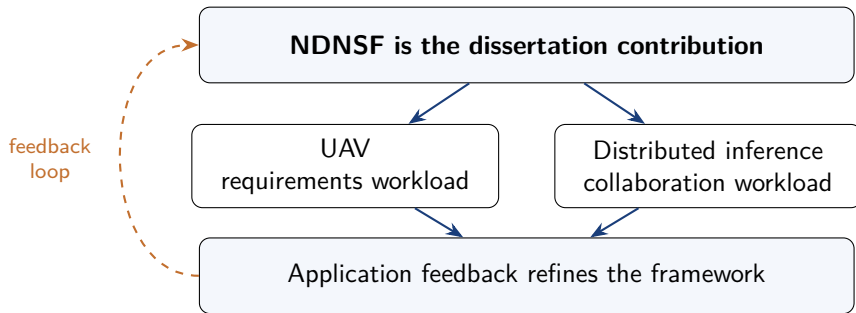
**Targeted collaboration gap.** NDN v0.1 already covers secure invocation, runtime Provider discovery and selection, authorization, replay protection, and reliable Data delivery.

The proposed extension addresses a narrower missing function: after runtime ACKs, derive a multi-role dependency plan and admit intermediate Data only when its name, signer, assigned role, dependency edge, and request context match, including Data retrieved from an NDN cache.

# NDNSF Contribution

| Framework function                                  | NDNSF contribution                                                                                                                                                                 |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Service interfaces and invocation</b><br>v0.1    | Generic C++ and Python APIs support Request, ACK, Selection, Response, and Targeted invocation without service-specific stubs; C++ also provides trusted same-process local calls. |
| <b>One-to-many service model</b><br>v0.1            | One named Request reaches willing Providers; applications use built-in or customizable Provider-selection strategies.                                                              |
| <b>Security and authorization</b><br>v0.1           | Name-based policy, NAC-ABE, signatures, one-time tokens, and replay checks protect invocation and Data.                                                                            |
| <b>Reliability and data delivery</b><br>implemented | SVS retry, segmented retrieval, large-object references, streaming, and bounded FEC recovery.                                                                                      |
| <b>Multi-role collaboration</b><br>proposed work    | Assign willing Providers to application roles, declare allowed Data flow, and verify Data received between assigned roles.                                                         |

# Dissertation Components



# Research Questions

- RQ1 Which service-layer functions are required to support secure invocation, dynamic multi-Provider selection, authorization, and reliable Data delivery over NDN?
- RQ2 How can NDN-SF assign Providers to application roles and verify that each downstream Provider accepts Data only from the Provider assigned to an authorized upstream role?
- RQ3 What reliability, security, and latency tradeoffs arise from multi-Provider publication, selection, Targeted invocation, and bounded response retry or Provider reselection?
- RQ4 How should mobility and control requirements in UAV network applications be supported through NDN-SF?
- RQ5 How should distributed inference use NDN-SF to reuse model files, assign model parts to available devices at runtime, and run one model across multiple devices?

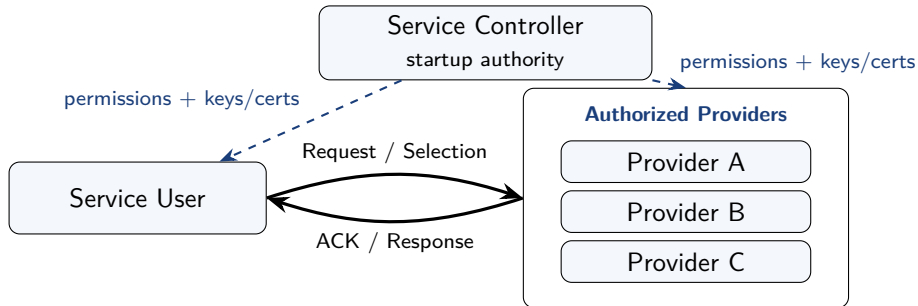
RQ2 tests role assignment and Data-flow verification for both directly received and cached Named Data.

# Proposed Work Status and Timeline

| Section                  | Completed Work                                                                                                | Remaining Work                                                                                       | Target         |
|--------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|----------------|
| NDNSF framework          | v0.1 invocation/security; Targeted invocation; trusted local calls; large-data support; execution-plan schema | Implement Plan-to-Data enforcement for direct and cached Data; measure security and latency overhead | Aug. 2026      |
| UAV workload             | Control, telemetry, video, and mobility workload                                                              | Multi-drone role/edge tests in MiniNDN [22] and PX4 SITL [23]                                        | Sept. 2026     |
| Distributed inference    | Planner, Repo, named dependency flow; preliminary model campaign                                              | Real-model and failure gates; final statistical evaluation                                           | Oct. 2026      |
| Evidence and chair draft | Proposal, survey, and v0.1 manuscript                                                                         | Freeze evidence; integrate chapters; submit chair draft                                              | Nov.–Dec. 2026 |
| Graduation path          | Spring 2027 path and deadlines confirmed                                                                      | Committee review; February 2027 defense; corrections and ProQuest                                    | Jan.–Mar. 2027 |

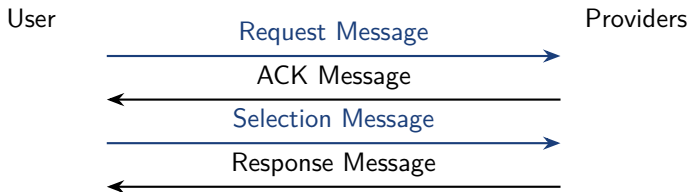
Defense by mid-Feb. 2027; ProQuest: March 5 target, March 26 deadline; Spring 2027 graduation.

# NDNSF Entity Roles



At startup, users and providers obtain permissions, keys, and certificates. During invocation, the user coordinates directly with providers through named Data messages.

# NDNSF Invocation Workflow



## Underlying NDN Sync [2]: announce, then fetch

1. Sender publishes each logical message as signed, named Data.
2. Sync announces the new Data name.
3. Receiver sends an Interest for that name and obtains the Data.

## SVS [3] Pub/Sub API: combines the two-step delivery

- ▶ `publish(name, payload)` stores the Data and announces its name.
- ▶ `subscribe(prefix, callback)` learns the name, fetches the Data, then invokes the callback.

**The NDNSF application does not manage the fetch itself.**

# NDNSF Access Control: Policy Examples

## Provider policy: Drone 1

```
provider /UAVNET/drone1
  allow /FlightControl/Takeoff
  allow /FlightControl/Land
  allow /ImageProc/YOLO12
```

## Provider policy: Drone 3

```
provider /UAVNET/drone3
  allow /FlightControl/Takeoff
  allow /Sensor/HyperSpectral
```

## User policy: Ground station

```
user /UAVNET/gs
  allow /UAVNET/drone1/
    FlightControl/Takeoff
  allow /UAVNET/drone1/
    ImageProc/YOLO12
  allow /UAVNET/drone3/
    Sensor/HyperSpectral
```

A Provider policy defines which services an identity may offer; a User policy defines which Provider/service namespaces the User may invoke. Neither depends on fixed endpoint addresses.

# Why ABE? Confidential Provider Discovery

**Reach authorized Providers before knowing who is willing and reachable.**

- ▶ The discovery descriptor states location, deadline, and resource needs for a positive-ACK decision.
- ▶ ABE [19] protects this content without enumerating Provider identities.
- ▶ Observers without the required decryption capability cannot read it; authorized but unselected Providers can.
- ▶ In the request-scoped path, Selection restricts application input and key material to selected Providers.

Trust Schema [16] alone does not encrypt content. Role authorization can be combined with ABE or service-group encryption. Outer Data names and traffic patterns remain visible.

# How ABE-Backed Authorization Checks Permission

**NAC-ABE [5,19] protects challenges; signatures identify participants.**

- ▶ Request carries a **Requester Challenge** (UserToken), protected for service-authorized Providers.
- ▶ ACK returns it and adds a **Provider Challenge** (ProviderToken), protected for service-authorized Users.
- ▶ Selection returns the chosen Provider's challenge.
- ▶ Each side validates signatures, service and request identity, challenge matching, and one-time state.

Challenges check access to ABE decryption capability. They do not replace signature validation or replay-state checks.

# Why ABE? Aggregate Permissions, Reuse Identity Credentials

| Authorization design                    | How service permissions are represented                                                                                                  |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| DNMP [12]                               | Role signing keys and Trust Schema rules constrain commands; roles may cover several permissions.                                        |
| Our per-service certificate alternative | Each entity receives a signing credential for each authorized service role.                                                              |
| NDNSF with KP-ABE [19]                  | One entity's DKEY aggregates service permissions within one authority and parameter generation; identity-signing credentials are reused. |

DKEY = decryption key. ABE avoids extra service-role signing credentials, not all key management. DKEY size and processing cost can grow with policy complexity; role designs can also reuse credentials.

# Runtime Revocation: What Changes and What It Costs

- ▶ **NDNSF**: validated Controller status makes runtime checks reject revoked invocations. ABE rotation protects future-generation exchanges.
- ▶ **DNMP [12]**: describes key rotation for future measurement-data access, not a detailed online command-revocation protocol.
- ▶ **Certificate alternative**: revoke the role credential; execution nodes check status and reject later commands.
- ▶ Both depend on update propagation and cannot retract disclosed keys or plaintext.

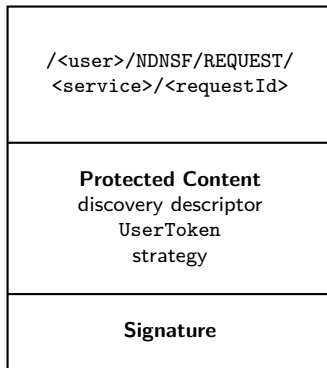
NDNSF: implemented and focused integration-tested. Controller-wide ABE rotation can require other authorized members to refresh DKEYs. This is supported functionality, not a uniquely ABE advantage.

# Request Data: Protected-Discovery Design

The descriptor lets authorized Providers decide whether to participate.

- ▶ Names the requested service.
- ▶ Identifies the requester and request instance.
- ▶ Carries the protected discovery descriptor.
- ▶ Includes a fresh Requester Challenge for this invocation.

**Carried as:** named, signed Data announced through SVS [3].



# Selective ACK

In the normal discovery path, every Provider returns an ACK.

- ▶ **Positive ACK:** the Provider is willing and currently capable.
- ▶ **Negative ACK:** the Provider received the request but is not ready or cannot serve it now.
- ▶ ACK metadata can report load, wait time, location, model availability, or safety state.
- ▶ User selects only among Providers with positive ACKs.

|                                                                                                   |
|---------------------------------------------------------------------------------------------------|
| <code>/&lt;provider&gt;/NDNSF/ACK/<br/>&lt;user&gt;/&lt;service&gt;/<br/>&lt;requestId&gt;</code> |
| <b>Content</b><br>ProviderToken<br>UserToken<br>ACK metadata                                      |
| <b>Signature</b>                                                                                  |

# Custom Provider Selection Strategy

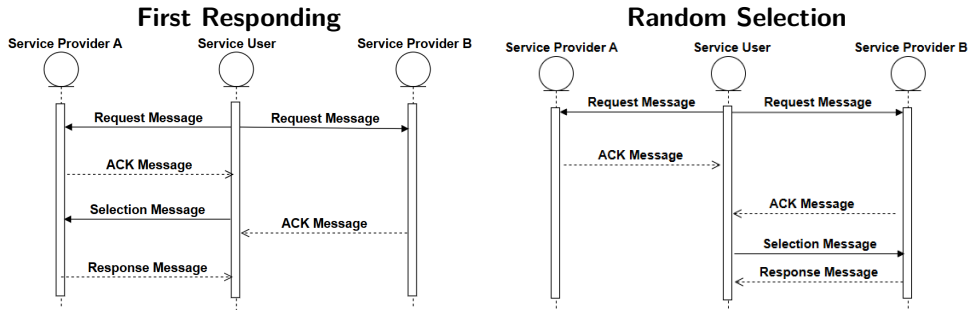
**Provider selection is user-side, policy-driven, and performed after ACK collection.**

- ▶ User collects all ACKs during an ACK window; only positive ACKs become selection candidates.
- ▶ Built-in choices: first responding, random, all selected.
- ▶ Custom policy can use latency, load, capability, location, or safety metadata.

|                                                                                                         |
|---------------------------------------------------------------------------------------------------------|
| <code>/&lt;user&gt;/NDNSF/<br/>SELECTION/&lt;provider&gt;/<br/>&lt;service&gt;/&lt;requestId&gt;</code> |
| <b>Content</b><br>ProviderToken<br>service input key                                                    |
| <b>Signature</b>                                                                                        |

**Key point:** ACK publication is common; ACK status reports readiness, and selection uses positive ACKs.

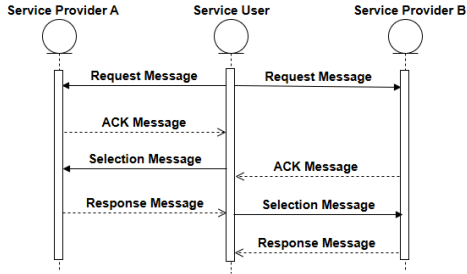
# Provider Selection Strategies: Built-In Choices



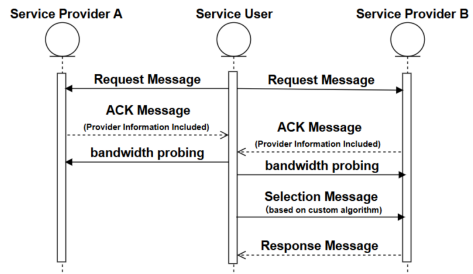
The same Request–ACK collection step can feed simple built-in policies: choose the first ready provider or pick randomly among valid ACKs.

# Provider Selection Strategies: Multi-Provider and Custom

## All Responders



## Custom Selection Strategy



Selection can also choose all ready providers for cooperative execution, or use application-defined metadata such as latency, load, capability, location, or safety state.

# Completed Foundation: NDNSF v0.1

- ▶ Implemented generic dynamic C++ API
- ▶ Multi-provider ACK collection
- ▶ Provider selection strategies
- ▶ NDN-SVS patch
- ▶ MiniNDN evaluation

## Provider API

```
provider.addHandler<RequestT, ResponseT>(
    serviceName, handler);
```

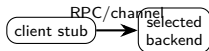
## User API

```
user.RequestService<RequestT, ResponseT>(
    providers, serviceName, request,
    onResponse, onTimeout, timeoutMs,
    strategy);
```

NDNSF v0.1 (commit `be9421f...`, tree `c9cc0c...`) is the May 20 repository baseline; role-based dependency collaboration is post-v0.1 work.

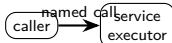
# Baselines Before the Comparison

## gRPC [17]



- ▶ Load balancing chooses among backend addresses supplied beforehand by a resolver; it does not discover willing Providers from an RPC.
- ▶ Here, gRPC-1 uses one endpoint; gRPC-SEQ-4 retries four preregistered endpoints sequentially.

## NSC [10]



- ▶ Expresses an RPC-like service call with NDN names.
- ▶ The evaluated configuration binds one call to one executor.

## NDNSF

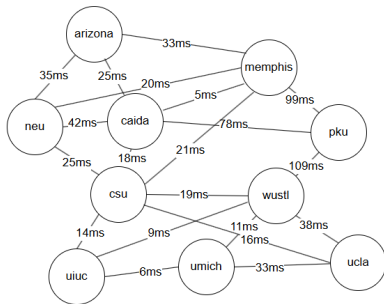


- ▶ Discovers multiple ready Providers through one ACK window.
- ▶ Selects after observing runtime willingness and metadata.

**Context only:** NDNSF uses submitted v0.1 measurements; NSC/gRPC use one new run per loss with the shown workload and deadline. Batches and semantics differ; no variance or significance claim is made.

# Network Loss: NDNSF v0.1 and Baselines

## Common topology and workload



**Path:** Memphis → CSU → UCLA; 37 ms one way.

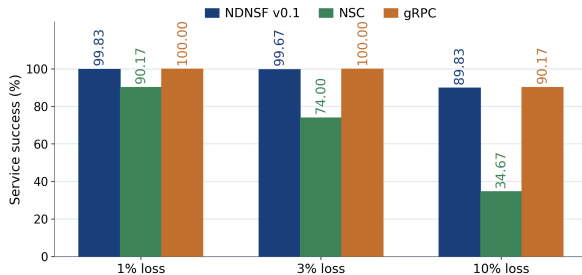
**Load:** 10 RPS; 5 ms service delay; 1/3/10% per-link loss.

**NDNSF run:** 60 s after warmup; 600 measured requests.

**Result:** With SVS retry, NDNSF completed 599/600, 598/600, and 539/600 requests at 1%, 3%, and 10% link loss.

## Service success rate (%)

Submitted NDNSF v0.1 values (10%: best measured); new NSC/gRPC: one run per loss. Configuration-aligned context only; no variance or significance claim.



# Historical v0.1: Provider Failures

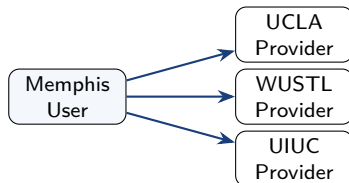
## Historical experiment

- ▶ Memphis User; identical service Providers at UCLA, WUSTL, and UIUC.
- ▶ Every 10 s, each Provider independently starts a 10 s rejection period with probability 10/20/30%.
- ▶ Unavailable NDNFS Providers suppress ACKs; FirstResponding selects a valid candidate.
- ▶ NSC and gRPC use one fixed target and no failover.

## What this established

- ▶ NDNFS remains above 96% success at the 30% setting.
- ▶ Motivation only; not a matched failover comparison.

## Historical multi-provider topology



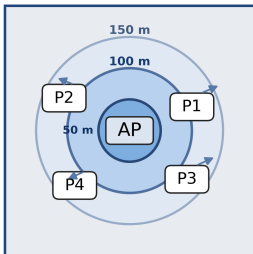
## Request success rate (%)

| System       | 10%          | 20%          | 30%          |
|--------------|--------------|--------------|--------------|
| NSC          | 83.25        | 66.58        | 49.92        |
| gRPC         | 83.35        | 66.69        | 65.36        |
| <b>NDNFS</b> | <b>99.82</b> | <b>99.46</b> | <b>96.70</b> |

**Evidence boundary:** historical configuration-level result; the baselines are not the four-target sequential clients evaluated on Slides 32–33.

# Four-Provider Mobility: Experiment Design

## One-AP field and evaluated radii



400 m  $\times$  400 m field; AP at (200, 200)  
Coverage radii: 50, 100, and 150 m; Provider speed: 2 m/s

400  $\times$  400 m field; AP at (200,200); 2 m/s random  
waypoint; 300 s simulated burn-in.

## Matched systems and controls

| System     | Client behavior                                     |
|------------|-----------------------------------------------------|
| NDNSF      | runtime ACKs; FirstResponding + bounded reselection |
| gRPC-SEQ-4 | four registered IPs; sequential 1 s retry           |
| NSC-4      | four registered prefixes; sequential 1 s retry      |

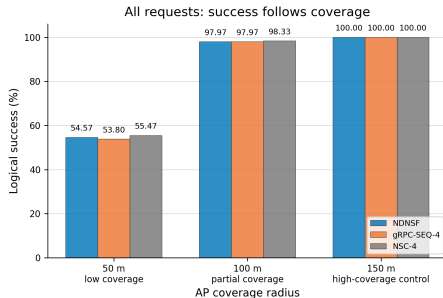
## Two matched analysis layers

- ▶ **All requests:** 50/100/150 m; coverage control.
- ▶ **Independent switching test:** 100 m; actual publication time and applied-gate agreement.
- ▶ 5 RPS, 60 s; 1 s attempt/ACK; 5 s deadline; admission and health routing disabled.
- ▶ `block_network=true`; same trace; 4.05 s request phase.

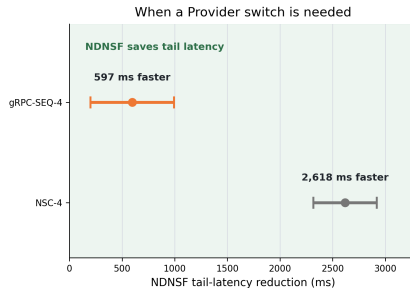
NDNSF does not require a pre-registered Provider endpoint list; gRPC and NSC receive the same four targets before each run.

# Mobility Results: Coverage and Switching

## Mobility controls and the independent switching test



**All requests:** At 50 m, all systems succeed on only about 54–55% of requests. At 100 m, success is about 98%; at 150 m, it reaches 100%. The main driver is which Providers are actually covered when a request is published.



**When switching is needed:** Among 1,312 matched requests where the initial Provider was unavailable but another Provider was reachable, NDNSF's tail latency was 597 ms lower than gRPC and 2.62 s lower than NSC. The intervals are shown in the chart. Success rates were 99.92/98.86/98.78% for NDNSF/gRPC/NSC; no overall success-rate advantage is claimed: actual Provider coverage at publication time dominates success.

# NDNSF v0.1 Evaluation: Availability

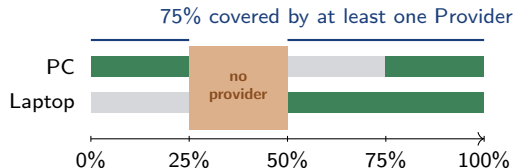
## Experiment settings

- ▶ Real-machine test: Odroid User with laptop and PC Providers.
- ▶ The laptop is unavailable during the first half; the PC is unavailable during the middle half.
- ▶ Their unavailable windows overlap for one quarter of the run.

## Observation

- ▶ At least one Provider is available for 75% of the run. This is the provider-time coverage ceiling, not a guaranteed success rate.
- ▶ NDNSF reaches 74.62% at 10 RPS and 73.73% at 30 RPS.
- ▶ The small gap below 75% is failover delay: an in-flight request can hit its deadline before another Provider is selected.
- ▶ SVS/Sync retransmission recovers lost Data, not Provider outages; recovery after the deadline still fails.

## Provider availability over time



## Measured request success (not time coverage)

TABLE VI: Service Availability in Real-World Experiments

| System       | 10 RPS        | 30 RPS        |
|--------------|---------------|---------------|
| NSC          | 52.67%        | 51.78%        |
| gRPC         | 49.33%        | 47.06%        |
| <b>NDNSF</b> | <b>74.62%</b> | <b>73.73%</b> |

# Mobility Evidence: What We Can Claim

## Historical v0.1 demonstrations

- ▶ Provider rejection and availability
- ▶ One NSC/gRPC target
- ▶ No seed-level inference
- ▶ **Role:** motivation only

## Matched all-request controls

- ▶ Same trace; four targets
- ▶ 50/100/150 m; repeated runs
- ▶ Coverage dominates success
- ▶ No universal advantage
- ▶ **Role:** negative controls

## Independent switching test

- ▶ 1,312 matched switching requests
- ▶ Tail latency: 597 ms below gRPC
- ▶ Tail latency: 2,618 ms below NSC
- ▶ Comparable success
- ▶ **Role:** confirmatory evidence

**Supported conclusion:** When a Provider switch is required, runtime discovery reduces tail latency versus sequential retry over pre-registered targets; it is not universally better under mobility.

# What NDNSF v0.1 Still Lacks

## Completed v0.1 evidence

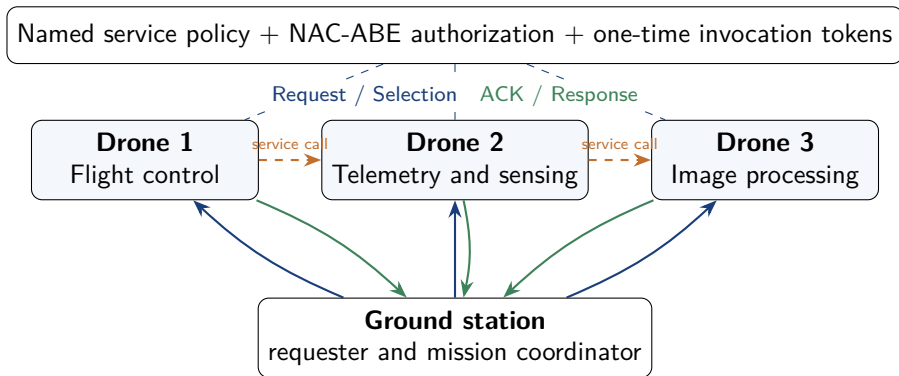
- ▶ Runtime discovery and service ACKs reporting willingness and status
- ▶ Provider selection under failure and mobility
- ▶ Request-level authorization and token binding

## Still missing for collaboration

- ▶ Assign different Providers to different application roles, such as capture, detection, and mapping.
- ▶ Record which role is allowed to send Data to which downstream role.
- ▶ Verify that received Data came from the Provider assigned to the authorized preceding role, whether fetched directly or from a cache.

UAV and distributed inference are requirement generators: they expose why Provider selection alone is insufficient for multi-role execution.

# UAV Workload



The ground station and drones may all invoke services; each drone can act as both an NDNFSF User and Provider during a mission.

# Requirements of a UAV Network Application

- ▶ Mobility and changing reachability
- ▶ Command authorization
- ▶ Telemetry freshness
- ▶ Video and recording objects
- ▶ Multi-drone missions

This dissertation will not produce a comprehensive UAV application. Instead, it will develop the NDN/SF mechanisms needed to support this class of application.

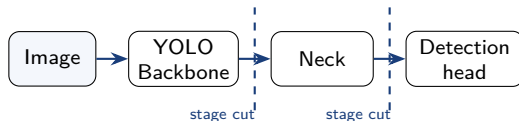
# UAV Requirements → NDNSF Mechanisms

| UAV requirement                    | NDNSF mechanism                                                                                                                                         |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mobility and changing reachability | Service ACKs report willingness and runtime status; NDNSF selects a responding Provider. Optional bounded response reselection may run after selection. |
| Command authorization              | Targeted invocation with NAC-ABE, one-time tokens, and replay checks                                                                                    |
| Telemetry freshness                | Versioned names, freshness/expiry checks, and lifecycle state                                                                                           |
| Video and recording objects        | Immutable object references, manifests, and segmented cache/Repo retrieval                                                                              |
| Multi-drone missions               | Assign Providers to mission roles; accept Data only from the Provider assigned to the authorized preceding role.                                        |

Generic one-to-many calls use ACK and Selection. Targeted calls skip them only after a Provider/service token batch has been bootstrapped.

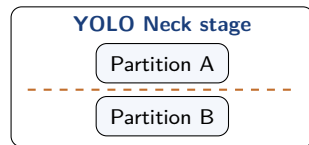
# Distributed Inference: Splitting a YOLO Model

**Layer-wise (horizontal) cuts:** contiguous layer groups form pipeline stages



Each stage owns a contiguous layer range and sends named intermediate activations to the next stage.

**Tensor-wise (vertical) cut:** one layer or stage is partitioned across devices



Each partition computes a slice of the same layer or stage; partial outputs may require exchange or aggregation.

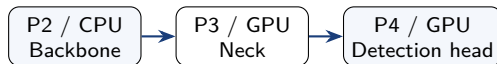
A hybrid layout combines layer-wise pipeline cuts with tensor-wise partitions; NDNFS assigns each stage or partition and device to the selected Providers.

# Distributed Inference Challenge

## Information available for one YOLO request

- ▶ signed NDN Data objects containing reusable YOLO layer artifacts;
- ▶ ONNX Runtime plus an NDN-SF-DI adapter that interprets and splits the model;
- ▶ Provider offers describing CPU/GPU, memory, load, and cached layers;
- ▶ the input image and deadline.

## Example collaboration plan



- ▶ P2, P3, and P4 receive only their assigned model parts and device instructions.
- ▶ Named intermediate tensors connect the assigned roles; each pipeline stage is a DI role.
- ▶ The plan is fixed only after signed Provider offers are collected.

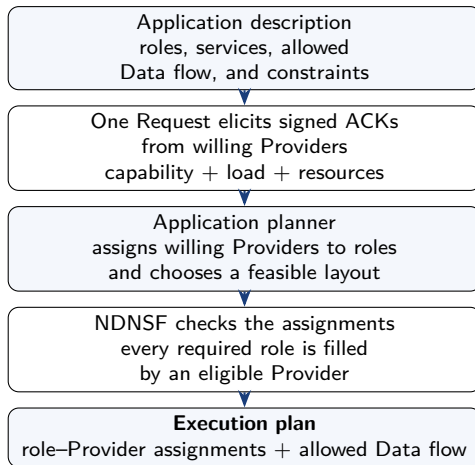
A proposed collaboration plan states which Provider runs each role, on which device, with which model part, and which Data may flow to the downstream role.

# DI Requirements → Proposed NDNF-DI Mechanisms

| Distributed-inference requirement                         | NDNF mechanism                                                                                              |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Run a model that does not fit on one available device     | Split the model into pipeline stages and, where supported, parallel tensor partitions                       |
| Use heterogeneous CPU and GPU Providers effectively       | Collect signed Provider offers, then choose Providers and devices after the ACK window closes               |
| Avoid copying a complete model for every request          | Reuse signed layer artifacts from an NDN Repo or cache; each Provider prepares only its assigned model part |
| Send intermediate tensors only along the planned pipeline | Assign Providers to roles and verify each tensor's request, signer, preceding role, and name                |
| Continue when a selected Provider becomes unavailable     | Use authenticated status and bounded replanning or Provider reselection within the request deadline         |

NDNF carries Request, ACK, Selection, and Data. Proposed NDNF-DI mechanisms interpret the model, choose the split, assign devices, and run each model part.

# Post-v0.1: From Application Roles to an Execution Plan



## Example after ACK collection

| Role      | Provider |
|-----------|----------|
| Capture   | UAV-1    |
| Detection | Edge-2   |
| Mapping   | GS-1     |

## Allowed Data flow

- ▶ Capture image → Detection
- ▶ Detected objects → Mapping

Only Providers that returned a valid ACK for this request may be assigned a role.

# Proposed: Verify Data Between Assigned Roles

## YOLO example from the execution plan



### Before P4 processes the tensor, it checks

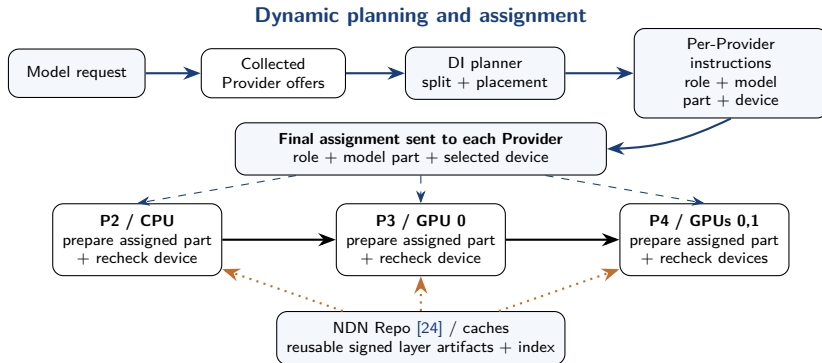
- ▶ it belongs to the current request;
- ▶ the signer is P2, assigned to Backbone;
- ▶ Backbone is the allowed preceding role;
- ▶ its name is under /tensor/features.

## Expected decisions

- ▶ **Accept:** correctly signed P2 feature tensor.
- ▶ **Accept:** the same valid tensor retrieved from an NDN cache.
- ▶ **Reject:** unassigned P3 uses the same request ID.
- ▶ **Reject:** P2 publishes on an undeclared Data name.
- ▶ **Reject:** a valid old tensor is replayed for another request.

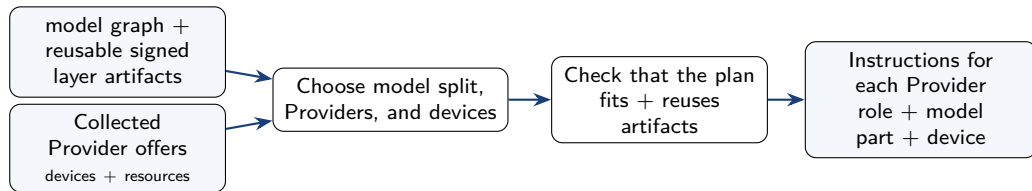
The proposed Plan-to-Data mechanism turns the execution plan into checks performed by each Provider.

# Proposed NDNF-DI Architecture



- ▶ NDNF Core remains model-neutral; the proposed NDNF-DI layer interprets the model, selects the split, and assigns Providers and devices.
- ▶ Providers prepare only their assigned model parts from reusable signed layer artifacts; the Repo does not store a new request-specific copy.
- ▶ A Provider offer reports current availability but does not reserve a GPU; each Provider rechecks resources immediately before loading.

# Proposed Inference Plan Generation



## Example decision

- ▶ stage 0 → P2 / CPU
- ▶ stage 1, partitions A–B → P4 / GPUs 0–1
- ▶ reuse signed layer artifacts already cached at P4

## Resulting execution contract

- ▶ Each Provider receives only its assigned role, model part, and device.
- ▶ Memory and topology are checked separately for every selected device.
- ▶ The same Data-flow checks apply to directly received and cached tensors.

Each signed Provider offer reports available devices, capacity, and cached signed layer artifacts.

# Pipeline, Tensor, and Hybrid Tradeoff

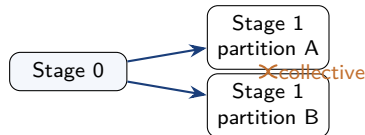
## Layer-wise pipeline cut: $N = 3$



- ▶ Assigns contiguous layer ranges to stages.
- ▶ Can span CPU and accelerator Providers.
- ▶ Transfers activations across stage boundaries.

## Hybrid: $N = 2$ , tensor partitions

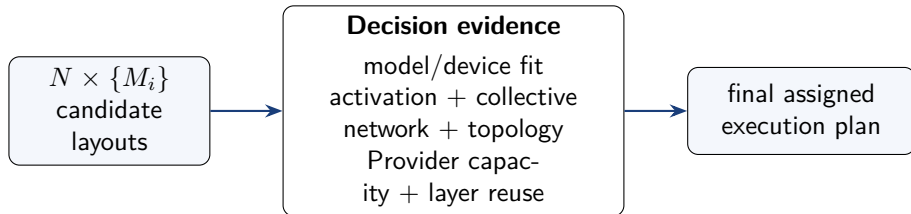
$$\{M_i\} = \{1, 2\}$$



- ▶ Shards selected stages across device sets.
- ▶ Requires adapter-certified tensor semantics.
- ▶ Adds collective and redistribution cost.

The planner chooses  $N \times \{M_i\}$  from model constraints, reuse, per-device feasibility, topology, activation size, and network cost.

# DI Planner Decision

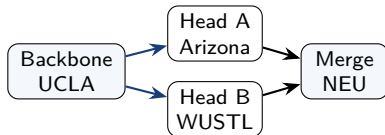


- ▶ Planning begins only after all timely ACKs are collected; an ACK does not reserve hardware.
- ▶ Every assigned model part must fit one actual device; memory is not combined across incompatible devices.
- ▶ Compatible cached layers may be reused after model and topology constraints pass.

# Preliminary Diagnostic Experiment

Earlier role/chunk prototype on the same full-network MiniNDN harness, using a small-model ONNX [25] workload instrumented by NativeTracer; 10 successful real-ONNX runs per layout.

## Shared-backbone layout



## Measured end-to-end latency

| Layout          | Mean  | p50   | p95   | SD   |
|-----------------|-------|-------|-------|------|
| Shared backbone | 259.4 | 267.7 | 283.9 | 22.1 |
| Single Provider | 186.8 | 191.7 | 206.3 | 20.0 |

All values are milliseconds.

Shared backbone uses four Providers; the single-Provider control runs all four roles at UCLA.

For this small model, the single-Provider layout is faster because it avoids fixed intermediate-data exchange. This diagnostic does not validate the proposed V3 reuse, hybrid partitioning, multi-GPU placement, or final resource recheck.

# Planned DI Validation

- ▶ **Identity and reuse:** signed layer-artifact, assembled-fragment, and loaded-runtime identities remain distinct and reproducible.
- ▶ **Placement/admission:** CPU and 0/1/2+ visible accelerators; per-device feasibility; no ACK-time reservation; JIT revalidation.
- ▶ **Execution modes:** pipeline, adapter-certified tensor, and heterogeneous  $N \times \{M_i\}$  hybrid cases.
- ▶ **Data/security:** dependency and collective edges, direct/cache equivalence, signer/name/key-scope negative tests.
- ▶ **Environments:** unit/integration harness first, then MiniNDN and sealed container/Tiger qualification.

Evidence remains gated: the V3 design is not presented as validated until model-file reuse, local assembly, device checks, and multi-device execution gates close.

# Planned Evaluation Matrix

| RQ          | Dissertation Component | Evaluation Target                                                                      | Evaluation Methodology                                         |
|-------------|------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------|
| RQ1         | Literature             | Prior-work boundary                                                                    | Literature and traceability analysis                           |
| RQ2–<br>RQ3 | Core                   | ACK-derived plan; role/edge admission; security, reliability, latency, overhead        | Protocol tests; direct/cache tests; matched baselines          |
| RQ4         | UAV                    | Mobility; authorization; freshness; media; collaboration                               | MiniNDN/PX4 SITL <a href="#">[23]</a> ; selected real machines |
| RQ5         | DI                     | Reuse model files; assign model parts at runtime; assemble locally; run across devices | Integration tests; MiniNDN; sealed real-model/container tests  |

Rows state planned evidence; the earlier small-model campaign is preliminary diagnostic evidence.

## **Dissertation thesis**

NDNSF can turn ACK-derived roles and named dependencies into enforceable policy for direct and cached Named Data.

The v0.1 implementation and initial evidence exist; Plan-to-Data enforcement and full validation remain proposed.

# Questions?

# Backup: Risk and Mitigation

| Risk                   | Mitigation                                            |
|------------------------|-------------------------------------------------------|
| UAV prototype scope    | Treat UAV as a requirements and validation workload   |
| Collaboration overhead | Measure ACK/Selection/Response and Targeted paths     |
| Large DI objects       | Use references, segmentation, and planner cost models |
| Variable hardware      | Keep MiniNDN/SITL as repeatable baselines             |

## Backup: UAV Validation Details

- ▶ **Targeted command lifecycle:** latency, timeout reason, explicit retry.
- ▶ **Typed state freshness:** telemetry, mission, video, and safety state.
- ▶ **Large media objects:** video and recording as signed segmented Data.
- ▶ **Mission recovery:** cancellation, partial recovery, and bounded role reassignment.

UAV validation asks whether NDNFSF mechanisms support mobile, command-sensitive service use cases.

## Backup: Runtime Metadata Evidence

### ► **Adaptive admission**

*Configuration:* open-loop 100 RPS offered load; compare no admission control with admission-aware NDNFSF.

*Result:* P95 latency stays at 268.12 ms with 0.00% timeout rate.

*Meaning:* providers should expose runtime capacity, not only service names.

### ► **Positive/negative ACK readiness**

*Configuration:* nearest provider is overloaded; the ACK-all baseline still selects it for 2209 of 2250 requests.

*Result:* success rises from 1.82% to 100.00% when negative ACKs exclude the overloaded provider.

*Meaning:* providers still return an ACK, but its status prevents an unavailable provider from becoming a selection candidate.

## Backup: User-Side Admission Control

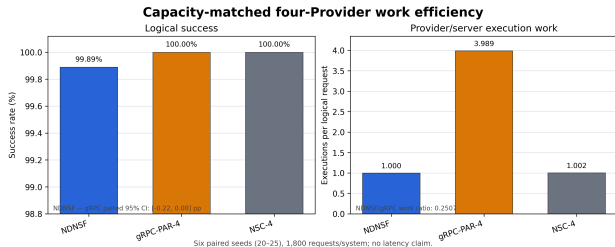
- ▶ NDNFSF bounds the number of outstanding service requests before they enter the Request–ACK–Selection–Response pipeline.
- ▶ Each request is locally admitted, queued, or rejected according to a window updated every 500 ms.
- ▶ The window grows when latency is stable, but shrinks under rising latency, timeouts, or near-full queues.
- ▶ Queue limits are derived from the current window, so overload protection adapts to runtime capacity instead of using one static threshold.

| 100 RPS offered load | Admission enabled          | Admission disabled           |
|----------------------|----------------------------|------------------------------|
| P95 latency          | 268.12 ms at 81 actual RPS | 4254.39 ms at 100 actual RPS |
| Timeout rate         | 0.00%                      | 3.55%                        |

# Backup: NDNFS Multi-Provider Coordination

The NDNFS paper treats provider coordination as a framework concern.

- ▶ One Request reaches multiple authorized providers for one named service.
- ▶ ACK metadata reports readiness; the user selects after ACK collection.
- ▶ Selected providers execute and publish Responses under NDNFS security.
- ▶ The mechanism is framework-level coordination, not application glue.



Capacity-matched, block-network gated: comparable success, 5,380 duplicate provider executions avoided. This supports work efficiency, not general mobility reliability or latency superiority.

# Backup: Why Not ABE Plus Role-Certificate Authorization?

**Both designs are valid; compare their permission-management costs.**

- ▶ **Alternative:** ABE protects discovery; service-role certificates and Trust Schema rules authorize execution.
- ▶ **NDNSF:** challenges reuse attribute-based permission management, avoiding extra service-role signing credentials.
- ▶ **Additional cost:** User permission DKEYs and challenge processing are not already paid for by Provider-side discovery.
- ▶ Both designs still validate signatures, request identity, current authority, and replay state.

Fewer credential objects do not prove lower total cost. Compare issuance, updates, key sizes, and revocation overhead under the same permissions.

## Backup: Middleware Comparison

**gRPC [17]** and **ROS 2 [18]** are useful, but they solve a different runtime problem.

- ▶ **gRPC:** strong RPC abstraction with name resolution, load balancing, and retry, but its channels carry messages between resolved service endpoints.
- ▶ **ROS 2:** strong robotics middleware built around a distributed robotics graph rather than NDN's named, cacheable Data model.
- ▶ **NDN advantage:** named, signed Data can be retrieved by name from providers, caches, or repositories.
- ▶ **NDNSF contribution:** adds authorization, ACK metadata, provider selection, replay protection, and large-data references above NDN.

NDNSF is not a replacement for every middleware stack; it studies what a data-centric service framework enables in dynamic, multi-provider environments.

# References [1–7]

- [1] L. Zhang et al., “Named Data Networking,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 66–73, 2014.
- [2] T. Li et al., “A Brief Introduction to NDN Dataset Synchronization (NDN Sync),” in *Proc. IEEE MILCOM*, pp. 612–618, 2018.
- [3] P. Moll et al., “A Brief Introduction to State Vector Sync,” Named Data Networking Project, Tech. Rep. NDN-0073, rev. 2, 2021.
- [4] Z. Zhang et al., “NAC: Automating Access Control via Named Data,” in *Proc. IEEE MILCOM*, 2018.
- [5] C. A. Lee et al., “Supporting Virtual Organizations Using Attribute-Based Encryption in Named Data Networking,” in *Proc. IEEE CIC*, pp. 188–196, 2018.
- [6] C. Tschudin and M. Sifalakis, “Named Functions and Cached Computations,” in *Proc. IEEE CCNC*, pp. 851–857, 2014.
- [7] M. Krol and I. Psaras, “NFaaS: Named Function as a Service,” in *Proc. ACM ICN*, pp. 134–144, 2017.

# References [8–13]

- [8] M. A. ur Rehman and B.-S. Kim, “CFEC: An Ultra-Low Latency Microservices-Based In-Network Computing Framework for Information-Centric IoVs,” *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3199–3212, 2024.
- [9] M. Krol et al., “RICE: Remote Method Invocation in ICN,” in *Proc. ACM ICN*, pp. 1–11, 2018.
- [10] D. Meirovitch and L. Zhang, “NSC—Named Service Calls, or a Remote Procedure Call for NDN,” Named Data Networking Project, Tech. Rep. NDN-0074, rev. 1, 2021.
- [11] M. Imran et al., “MIA-NDN: Microservice-Centric Interest Aggregation in Named Data Networking,” *Sensors*, vol. 23, no. 3, Art. no. 1411, 2023.
- [12] K. Nichols, “Lessons Learned Building a Secure Network Measurement Framework Using Basic NDN,” in *Proc. ACM ICN*, pp. 112–122, 2019.
- [13] R. Tourani et al., “Towards Security-as-a-Service in Multi-Access Edge,” in *Proc. ACM/IEEE SEC*, pp. 358–363, 2019.

# References [14–21]

- [14] M. Król et al., “Compute First Networking: Distributed Computing Meets ICN,” *ACM ICN*, 2019.
- [15] C. Barrios et al., “CaSCON: Cache-Assisted Service Composition Orchestration Leveraging NDN,” *IEEE SMARTCOMP*, 2025.
- [16] Y. Yu et al., “Schematizing Trust in Named Data Networking,” *ACM ICN*, 2015.
- [17] gRPC Authors, “Core Concepts, Architecture and Lifecycle,” gRPC Documentation, accessed July 2026.
- [18] S. Macenski et al., “Robot Operating System 2: Design, Architecture, and Uses in the Wild,” *Science Robotics*, 2022.
- [19] S. Dulal et al., “Enhancing NAC-ABE to Support Access Control for mHealth Applications and Beyond,” *arXiv:2311.07299*, 2023.
- [20] C. Marxer et al., “Access-Controlled In-Network Processing of Named Data,” *ACM ICN*, 2016.
- [21] S. D. Fu et al., “Toward Data-Centric Service Composition,” *ACM HotNets*, 2024.

# References [22–25]

- [22] Mini-NDN Developers, “Mini-NDN: A Mininet-Based NDN Emulator,” project documentation, accessed August 2026.
- [23] PX4 Development Team, “Simulation: Software in the Loop (SITL),” *PX4 User Guide*, accessed August 2026.
- [24] Named Data Networking Project, “Repo Protocol Specification,” repo-ng documentation, accessed August 2026.
- [25] ONNX Project, “Open Neural Network Exchange Intermediate Representation Specification,” ONNX documentation, accessed August 2026.